



Fake it 'till you make it

**Zarr-like Access of Existing NetCDF4
Datasets**

Lucas Sterzinger

Mentors: Chelle Gentemann (Farallon Institute), Julia Kent (NCAR), Kevin Paul (NCAR)

July 28, 2021



The Problem

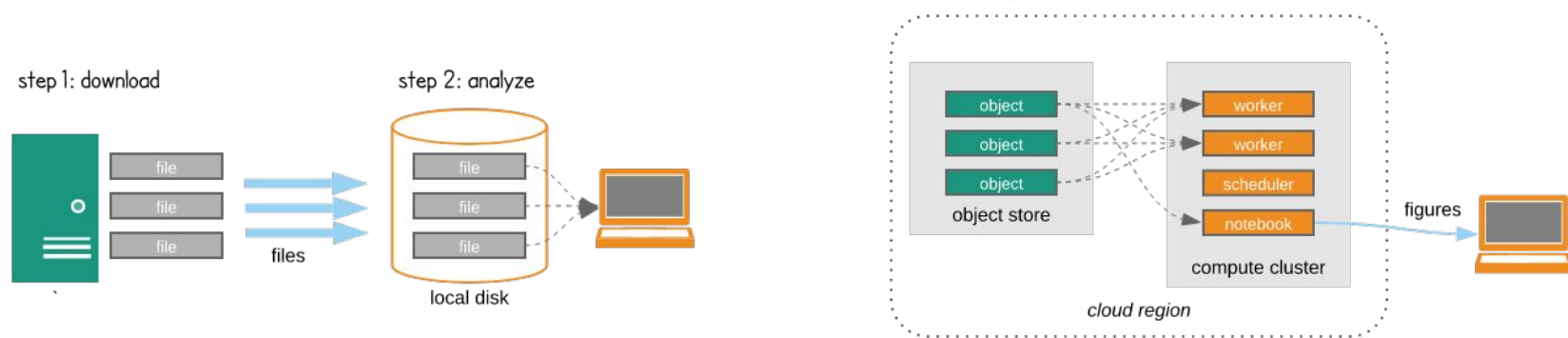
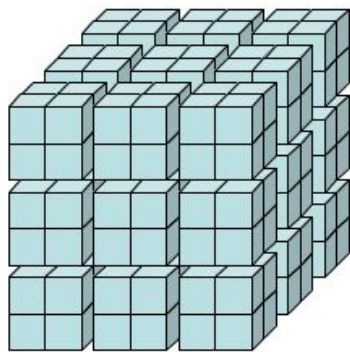


Image Source - Pangeo/Ryan Abernathy

- Peta(Exa?)bytes of geophysical data now available in the cloud
- Much of the data are in NetCDF4/HDF5 format
 - Not efficient for cloud computing

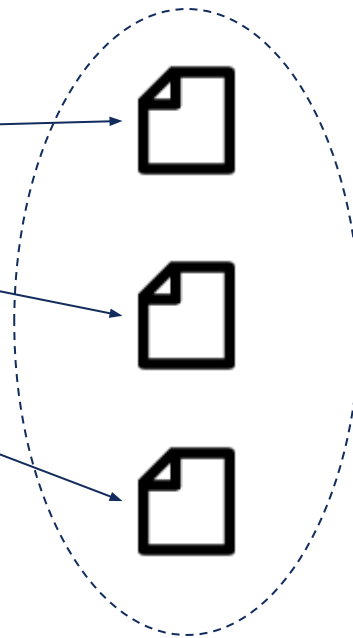
NetCDF vs Zarr

NetCDF



Single File Object

Zarr



Multiple File Objects
(One per chunk)



Plaintext
Metadata
File

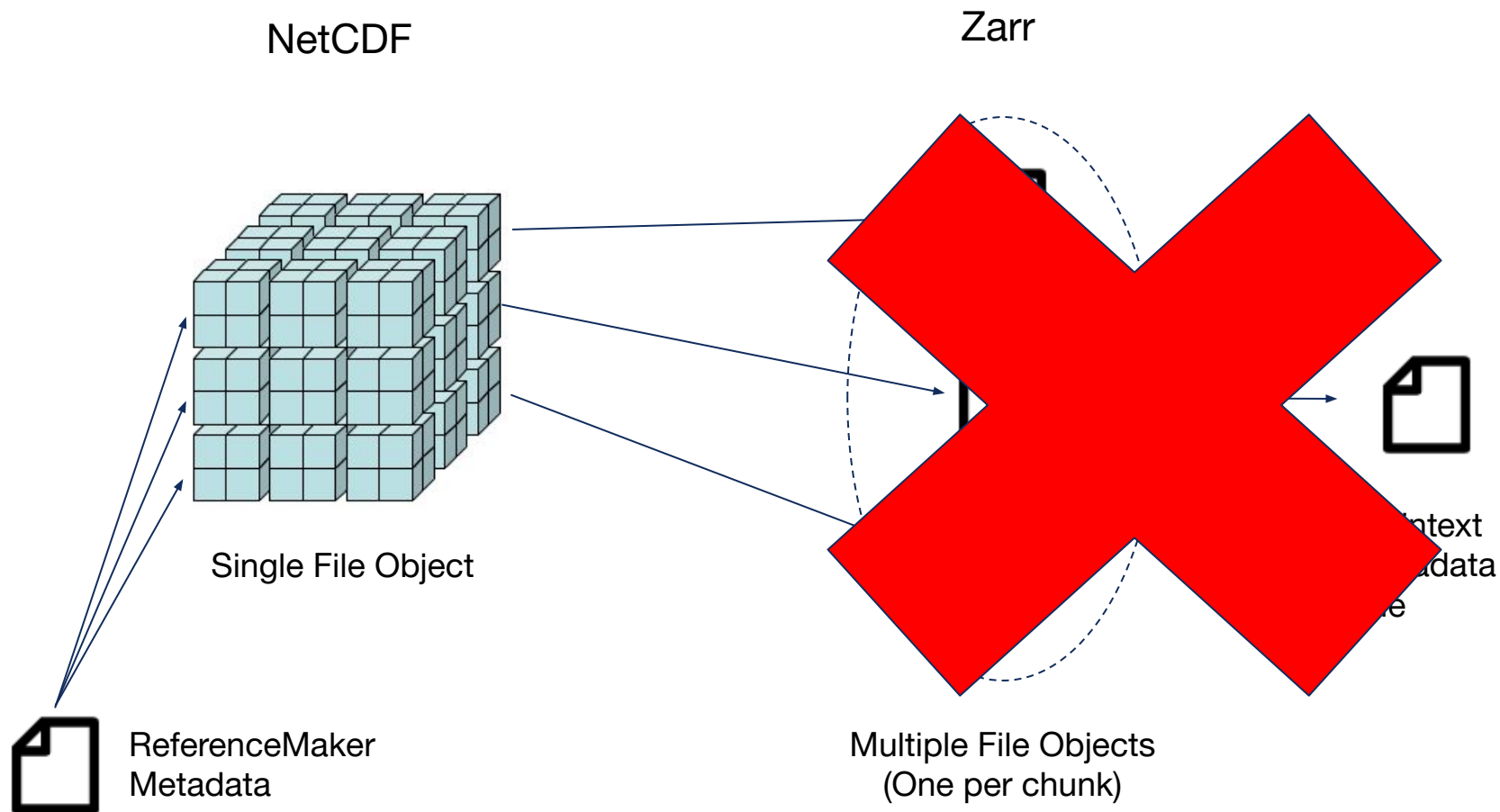
How does it work?

ReferenceMaker/ReferenceFileSystem:

- Part of Intake group's `fsspec` project
- Access existing NetCDF/HDF files *as if they are Zarr*
- Zarr-like Metadata files (few MB each)
 - Remote file address (Azure Blob, AWS S3, etc)
 - Data variable and chunking info
 - Can be read as a Zarr file
- Extracts byte-ranges for individual variables/chunks
 - Instead of individual chunk files in Zarr



ReferenceMaker



Creating JSONS

`@dask.delayed` ←

Easily distributed with Dask!

```
def gen_json(url):  
    so = dict(  
        mode="rb", anon=True, default_fill_cache=False, default_cache_type="none"  
    )
```

```
    outpath = './jsons/' + url.split('/')[-1].split('.nc')[0] + '.json'
```

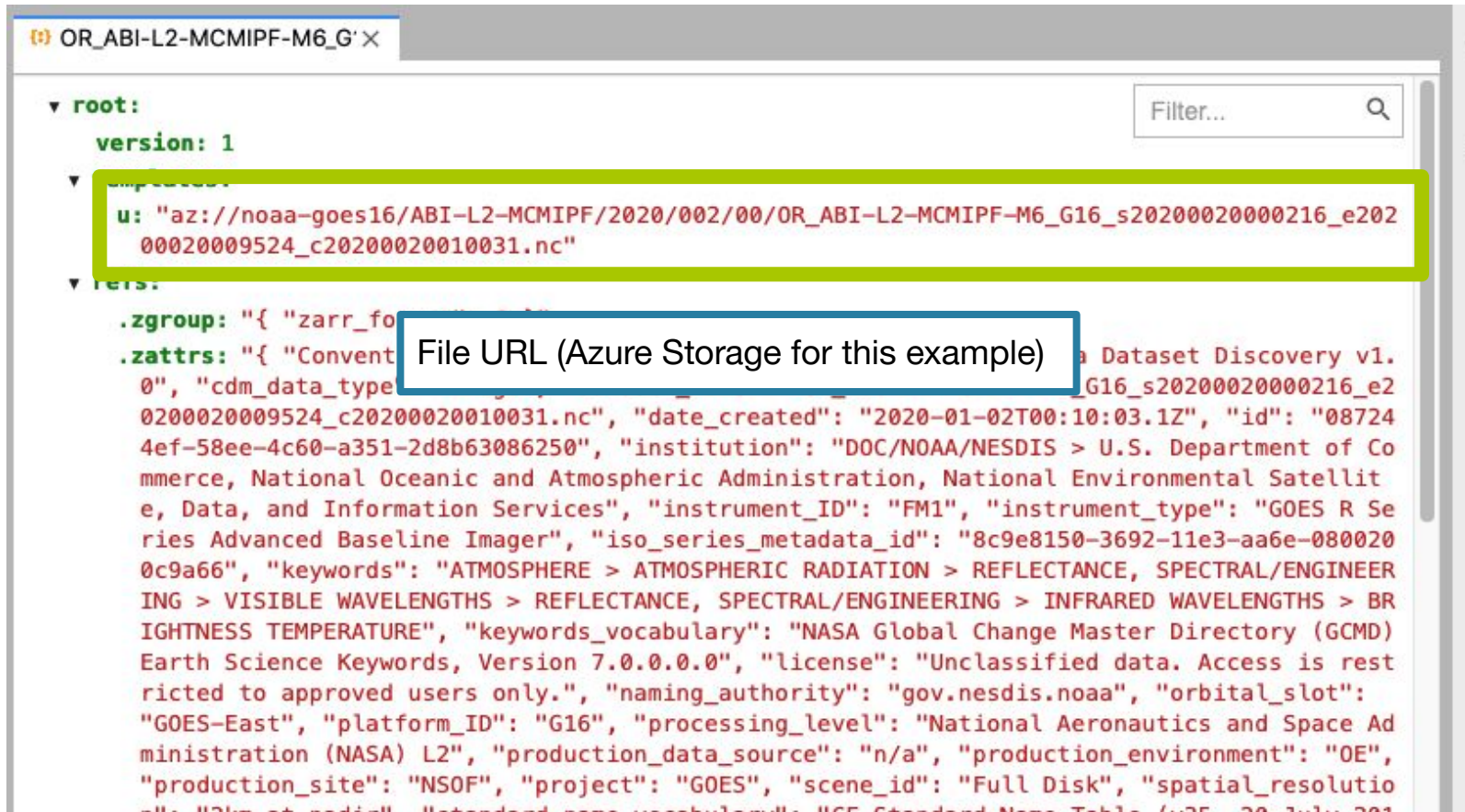
```
    with fsspec.open(url, **so, account_name='goesewest') as inf:  
        h5chunks = SingleHdf5ToZarr(inf, u, inline_threshold=300)
```

Read remote file and
process chunks

```
    with open("./reference.json", 'wb') as outf:  
        outf.write(ujson.dumps(h5chunks.translate()).encode())
```

Write metadata to file

Reference JSON Example



Variable (CMI_C01) chunk info,
compression, shape, etc

```
CMI_C01/.zarray: "{ \"chunks\": [ 226, 226 ], \"compressor\": { \"id\": \"zlib\", \"level\": 1 }, \"dt  
type\": \"<i2\", \"fill_value\": -1, \"filters\": null, \"order\": \"C\", \"shape\": [ 5424, 5424 ], \"z  
arr format\": 2 }"
```

```
CM1_C01/.zattrs: "{ \"_ARRAY_DIMENSIONS\": [ \"y\", \"x\" ], \"_FillValue\": -1, \"_Netcdf4Dimid\": 0, \"_Unsigned\": \"true\", \"add_offset\": 0.0, \"ancillary_variables\": \"DQF_C01\", \"cell_methods\": \"t: point area: sum (interval: 0.000028 rad)\", \"coordinates\": \"band_id_C01 band_wavelength_C01 t y x\", \"downsampling_method\": \"average\", \"grid_mapping\": \"goes_imager_projection\", \"long_name\": \"ABI Cloud and Moisture Imagery reflectance factor\", \"resolution\": \"y: 0.000056 rad x: 0.000056 rad\", \"scale_factor\": 0.00031746001332066953, \"sensor_band_bit_depth\": 10, \"standard_name\": \"toa_lambertian_equivalent_albedo_multiplied_by_cosine_solar_zenith_angle\", \"units\": \"1\", \"valid_range\": [ 0, 4095 ] }"
```

Single Chunk Byte Range

Reference JSON Example

Specify filesystem type
“reference” and JSON

```
fs = fsspec.filesystem('reference',  
                        fo='reference.json',  
                        remote_protocol='az',  
                        remote_options={'account_name': 'goeseuwest'},  
                        simple_templates=True)
```

Specify remote file
protocol (Azure here) and
authentication

```
m = fs.get_mapper('')  
ds = xr.open_dataset(m, engine='zarr|')
```

Pass mapper to xarray and
specify Zarr engine

GOES Workflow



Data stored in same region as compute

A banner image for the Microsoft Planetary Computer, showing a bright sun rising over a blue horizon with a dark sky.

Microsoft Planetary
Computer

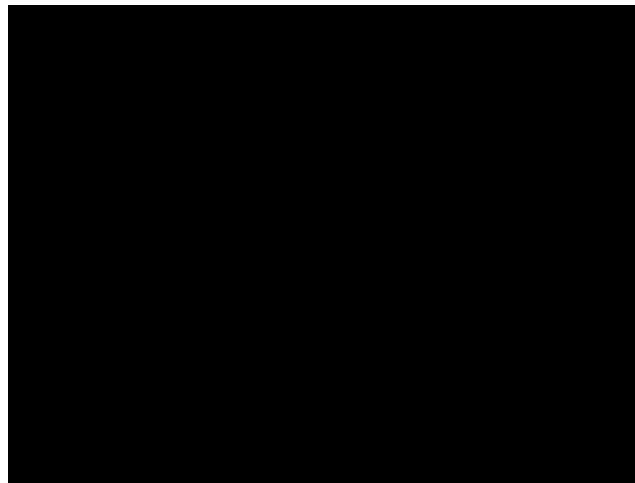
4 CPU, 32 GB Memory,
8 Dask Workers

- Created a test workflow on Microsoft Planetary Computer
- Analysis of 24 hours of GOES data
- Test access to data stored in Azure in 3 formats:
 - NetCDF4
 - NetCDF4 + ReferenceMaker
 - Zarr

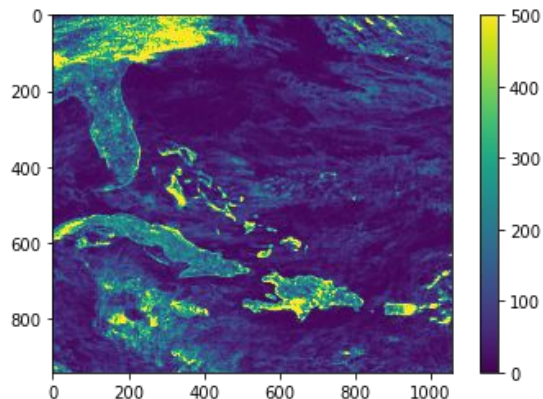
GOES Workflow



Data stored in same region as compute



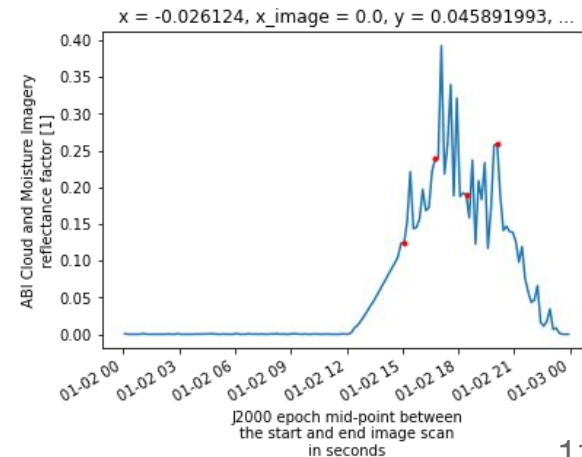
4 CPU, 32 GB Memory,
8 Dask Workers



True Color 2020-01-02 0005 UTC



Veggie Color 2020-01-02 0005 UTC



GOES Processing Times

For 24 hours of GOES-16 “ABI-L2-MCMIPF” product data

<u>Format</u>	<u>Preprocess Time</u>	<u>Data Open Time</u>	<u>Workflow Time</u>	<u>Extra Storage</u>
Remote NetCDF4	0 min	10 minutes	40 min	0 GB
Zarr	1 h 38 min	30 seconds	4 min	52 GB
ReferenceFileSystem	1 h 25 min	35 seconds	5 min 30 s	<u>416 MB</u>

- NetCDF has no upfront preprocess time cost
 - But very slow open and workflow times
- Zarr comes at cost of high storage costs (full data duplication)
- ReferenceFileSystem has speed of native Zarr but orders of magnitude less storage cost

This is a big deal!

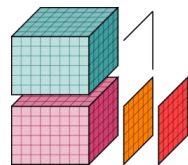
- ReferenceMaker/FileSystem allows users to harness the cloud-optimization of Zarr without needing to convert any data
- Allows current institutions to cheaply make existing cloud-hosted data more optimized
- Reference files can be also created and hosted by third parties
- Automatable
 - e.g. Pangeo Forge

If you are a scientist that works with “Big Data”

- Join the Pangeo community
- Talk to project maintainers about your workflow needs
- Open issues/bug reports on GitHub
- Submit pull requests, even if your code isn't 100% ready

PANGEO

Microsoft Planetary
Computer



xarray



Iris



Acknowledgements

Thank you to the ReferenceFileSystem team!

- Special thanks to Martin Durant (Anaconda) and Rich Signell (USGS) for their help learning and contributing to this project
- <https://github.com/intake/fsspec-reference-maker>

Thank you to Kevin Paul (NCAR), Julia Kent (NCAR), and Chelle Gentemann (Farallon Institute) for their amazing mentorship through the SIParCS program

Thanks to the SIParCS team for working so hard to make a fully-online internship experience so successful

Thanks to Microsoft for providing credits for Planetary Computer

Please feel free to reach out with questions
lsterzinger@ucdavis.edu or twitter [@lucassterzinger](https://twitter.com/lucassterzinger)



Link to site with interactive code examples, tutorials, resources, and contact info

<https://lucassterzinger.com/2021-siparcs-poster/>